Usernode Waitlist API

Integration reference for third-party websites

Version: September 2026 • Base URL: `https://social-vibecoding.usernodelabs.org`

Overview

These endpoints power the platform's public waitlist. An external website can reproduce the full signup journey server-to-server:

1. A visitor types their email (plus, optionally, how they found the platform and where they live) and joins the waitlist. Joining needs no account and no login.
2. They receive a confirmation email containing a one-click confirm link and a six-digit code. Either one confirms their address; the link also drops them straight into the optional "Want in sooner?" survey.
3. On that survey they can share a link to something they made, describe their group, connect a GitHub, X, or LinkedIn account to prove it is theirs, and copy a personal invite link. Friends who join through that link are counted and shown back to them with masked email addresses.
4. Re-submitting the same email later is silently accepted, so nobody can use the form to test whether an address is already on the list.

General rules

- **Base URL:** the platform origin, `https://social-vibecoding.usernodelabs.org` by default. A self-hosted deployment substitutes its own configured domain.
- **Authentication:** none. Every endpoint here is anonymous. The only credential in the flow is the `more_token`, a 48-character lowercase hex capability string returned by the first join and carried in the confirmation email. Treat it like a password for that signup: whoever holds it can read and edit the survey answers.
- **Content type:** send `Content-Type: application/json` on all POSTs.
- **CORS:** the server sends no `Access-Control-Allow-Origin` headers, so browser fetch calls from a third-party origin will be blocked. Integrate server-to-server: your backend forwards the visitor's submission to the platform and relays the response.
- **Errors:** every non-2xx response is JSON of the shape `{ "error": "human readable message" }`. Show `error` to the visitor as-is; the messages are written for end users.
- **Rate limits** (per client IP):
  - Join: 5 requests per 15 minutes per IP.
  - All token-based routes (confirm, survey read, survey save): 60 requests per 15 minutes per IP, shared across those routes.
  - A throttled request returns HTTP 429 with `{ "error": "...", "retryAfterSeconds": 540 }` plus draft-7 `RateLimit` response headers. IPv6 clients are bucketed by /56 prefix.

1. Read the form definitions

`GET /api/public/waitlist/options` — no rate limit

No auth, no rate limit, static per process. Returns every option key and display label the forms use. Render your form from this so your labels and the server's validation cannot drift; the server rejects any key not in this payload.

`200` Response (abbreviated):

```
{
  "discovery_sources": { "x": "X", "linkedin": "LinkedIn", "instagram": "Instagram",
                         "reddit": "Reddit or a forum", "friend": "Friend or colleague",
                         "podcast": "Podcast", "event": "Event", "other": "Other" },
  "group_sizes":  { "lt10": "Under 10", "...": "..." },
  "group_roles":  { "founder": "I started it", "...": "..." },
  "group_tools":  { "discord": "Discord", "...": "..." },
  "loss_answers": { "yes": "Yes, and it still annoys me", "...": "..." },
  "loss_kinds":   { "shutdown": "Shut down for good", "...": "..." },
  "countries":    { "North America": { "US": "United States", "...": "..." },
                   "Europe": { "...": "..." } }
}
```

`countries` is grouped by region; the keys of each region object are the accepted 2-letter codes (mostly ISO-3166 alpha-2, plus regional pseudo-codes `EU`, `LA`, `AF`, `ME`, `AP` meaning "elsewhere in that region").

2. Join the waitlist

`POST /api/public/waitlist` 5 per 15 min per IP

FIELD	TYPE	REQUIRED	RULES
`email`	string	yes	Trimmed, lowercased, max 255 chars, must match a basic `local@domain.tld` shape
`discovery_source`	string	no	One of the keys in `discovery_sources`
`country`	string	no	One of the 2-letter codes in `countries` (case-insensitive, stored uppercase)
`invite_code`	string	no	The 10-character code from a `/#waitlist?ref=CODE` invite link. A stale or invalid code is silently ignored, never an error

Unknown fields are dropped, not rejected. An email-only body is a valid signup.

Example request:

```
{ "email": "dev@example.com", "discovery_source": "podcast",
  "country": "DE", "invite_code": "a1b2c3d4e5" }
```

`200` First-ever join for this email:

```
{ "ok": true,
  "message": "You're on the waitlist. We'll email you when access opens up.",
  "more_token": "3f7a...48 hex chars...c9" }
```

`200` Email already on the list: identical, except `"more_token": null`. The endpoint is deliberately non-enumerating: you cannot tell "already joined" apart from "just joined" except by the token, and the token only ever goes to the first join. Store `more_token` if you want to open the survey for the visitor immediately; it is also emailed to them.

`422` `{ "error": "A valid email address is required." }` or `{ "error": "Unknown discovery source." }` or `{ "error": "Unknown country." }`

Side effect on first join: the platform emails the address a one-click confirm link (`/api/public/waitlist/confirm/<more_token>`) and a six-digit code. Mail is best-effort; the join succeeds even if mail delivery is not configured.

3a. Confirm by emailed link (browser navigation, not an API call)

`GET /api/public/waitlist/confirm/:token` 60 per 15 min per IP

`:token` is the 48-hex `more_token`. Stamps the signup as confirmed (idempotent, first timestamp wins) and answers `302 Location: /#more/<token>` so the visitor lands on the survey. Unknown or malformed token: `404 { "error": "Unknown or expired link." }`. You normally never call this yourself; the email carries it. State change over GET is deliberate: the token is an unguessable capability delivered to the address being confirmed.

3b. Confirm by six-digit code

`POST /api/public/waitlist/confirm` 60 per 15 min per IP

For an "enter the code from your email" UI, useful on phones where leaving for the mail client loses the visitor's place.

Request body:

```
{ "email": "dev@example.com", "code": "482913" }
```

`code` must be exactly six digits as a string. Codes expire 15 minutes after the join and allow 5 wrong attempts; only the newest code per email is live.

`200` Response:

```
{ "ok": true, "more_token": "3f7a...c9" }
```

This is the second way to obtain the `more_token` (for example when the visitor returns on a different device).

`422` One message for every failure (wrong code, expired, too many attempts, email never joined), again to prevent enumeration:

```
{ "error": "That code is wrong or has expired. Ask for a new one." }
```

A missing or malformed input gets `{ "error": "Enter the six-digit code from your email." }` instead. There is no public "resend code" endpoint; a fresh code is only minted on a first join.

4. Read the survey state

`GET /api/public/waitlist/more/:token` 60 per 15 min per IP

Returns previously saved answers (for prefilling), which OAuth connect buttons to show, the follow links, and the visitor's personal invite link (minted on first read of this endpoint).

`200` Response:

```
{
  "ok": true,
  "answers": {
    "_version": 3,
    "discovery": { "source": "podcast" },
    "country": "DE",
    "made_url": "https://example.com/project",
    "group": { "name": "Berlin Makers", "size": "10-50", "role": "organizer",
               "tools": ["discord"], "need": "..." },
    "loss": { "had": "yes", "product": "OldForum", "kind": ["shutdown"], "story": "..." },
    "handles": { "discord": "dev#1234" },
    "verified": { "github": "octocat" },
    "admit_together": true,
    "followed_claim": false
  },
  "oauth":  { "github": true, "x": true, "linkedin": false },
  "follow": { "x": "https://x.com/...", "linkedin": null, "instagram": null },
  "invite": { "url": "https://social-vibecoding.usernodelabs.org/#waitlist?ref=a1b2c3d4e5",
              "count": 2, "emails": ["jo***@example.com", "sa***@mail.net"] }
}
```

`answers` contains only sections the visitor has saved (a fresh signup with no survey answers returns `{}` or just the stage-1 keys). An `oauth` provider flag of `false` means that connect button should render as a plain text input instead. A `follow` entry of `null` means render no link for that network. `invite.emails` are always masked.

`404` `{ "error": "Unknown or expired link." }`

5. Save survey answers

`POST /api/public/waitlist/more/:token` 60 per 15 min per IP

Everything is optional. Saving merges section-wise: a section you send replaces that whole section, sections you omit keep their previous value, so the form can be filled over several visits. Unknown fields are dropped; unknown enum values are a 422.

Request body fields (flat, the server assembles the nested sections):

FIELD	TYPE	RULES
`made_url`	string	Max 2000 chars, must start `http://` or `https://` and contain a dot, else 422 "That does not look like a link. It should start with https://"
`made_note`	string	Max 140 chars, only stored when `made_url` is present
`group_name`	string	Max 255
`group_size`	string	Key from `group_sizes`
`group_role`	string	Key from `group_roles`
`group_tools`	array of strings	Each a key from `group_tools`, else 422; non-array is 422 "group_tools must be a list."
`group_need`	string	Max 800
`had_loss`	string	Key from `loss_answers`
`loss_product`	string	Max 255
`loss_kind`	array of strings	Each a key from `loss_kinds`
`loss_story`	string	Max 800
`farcaster`, `discord`, `telegram`, `other_handle`	string	Max 255 each; self-reported handles
`admit_together`	boolean	Coerced with truthiness
`followed_claim`	boolean	A claim only; the platform never verifies follows

Strings over their max length are treated as absent, not rejected. Empty strings are treated as absent.

Example request:

```
{
  "made_url": "https://example.com/project",
  "made_note": "A community tool I built",
  "group_name": "Berlin Makers",
  "group_size": "10-50",
  "group_role": "organizer",
  "group_tools": ["discord", "spreadsheet"],
  "had_loss": "yes",
  "loss_kind": ["shutdown"],
  "admit_together": true
}
```

`200` Response:

```
{ "ok": true, "message": "Saved, thanks. You can come back and add to this any time." }
```

`404` `{ "error": "Unknown or expired link." }` `422` The specific validation message from the table above.

6. Social connect (browser redirect flow, optional)

`GET /waitlist/connect/:provider?token=<more_token>`

`:provider` is `github`, `x`, or `linkedin`. Not an XHR endpoint: navigate the visitor's browser (or open a window) to it. The server validates the token, then 302s to the provider's OAuth authorize page; the provider calls back to `GET /waitlist/connect/:provider/callback`, and the visitor finally lands on the platform's own survey page at `/#more/<token>?connect=<status>` with `status` one of `ok`, `denied`, `failed`, or `unavailable`.

The verified handle appears afterwards under `answers.verified.<provider>` on the survey read endpoint. Only offer the button for providers whose `oauth` flag was `true`. Because the flow ends on the platform origin, a third-party site should treat it as a hand-off, not something it can round-trip back to its own pages.

Invalid token on the start route redirects to `/#landing` rather than erroring. Callbacks are replay-safe: reloading the callback URL re-issues the same final redirect.

Integration notes

- The join limiter (5 per 15 min) is keyed by client IP. A third-party backend proxying many visitors through one egress IP will hit it quickly; the platform trusts forwarding headers only from its own proxy, so an external integrator's forwarded headers are not honored. High-volume integrators would need a platform-side allowance, which does not exist today.
- The `invite.emails` masking, the non-enumeration contract (identical responses for known and unknown emails on join and code confirm), and the token-as-capability model are deliberate security properties. Do not "improve" them away on your side.
- The fields `discovery_detail`, `city`, and `referrer_handle` (stage 1) and the `invites` array (stage 2) are retired: a client still sending them succeeds with the keys silently dropped. Do not expect them to persist if you copy an older form.
- Admin-side waitlist endpoints (release and review) require an admin session and are not part of this integration surface.

Usernode Social Vibecoding. Waitlist API integration reference, generated September 2026. Messages quoted in responses are the exact strings the API returns and are safe to show to end users.